

Test Plan

Campus Equipment Booking & Return System

Course: Software Testing and Validation - Addis Ababa University **Group members:** (add names and student IDs here)

1. Introduction / Purpose

This plan covers testing an existing Flask application (Campus Equipment Booking & Return System) that was audited, had real defects fixed, gained a missing feature, and then received a full automated test suite. The purpose is both to verify the application's business rules and to demonstrate every technique required by the course using real, executed evidence rather than documentation written in isolation from the code.

2. Scope

In scope: registration/login/roles, equipment catalogue and maintenance status, the full booking lifecycle (request, approve/reject, activate/hand-over, return, late-flagging, cancellation), availability and conflict checking, and the CI pipelines that run the suite.

Out of scope: payment, notifications, multi-campus support, load testing, accessibility auditing beyond semantic HTML.

3. Product overview

See the README for full architecture. In short: Flask + SQLAlchemy + Flask-Login, two roles (student/admin), SQLite storage.

4. Objectives

- Verify every business rule in the requirements table (Test Design Document section 1).
- Reach $\geq 80\%$ branch coverage on core business logic (achieved: 99% on `booking_service.py`, 100% on `auth.py`).
- Demonstrate the test pyramid: 88 unit tests, 27 integration tests, 12 E2E tests.
- Find and fix real defects rather than only writing passing tests around existing behaviour (4 defects found and fixed - see Defect Log).
- Demonstrate a real regression caught by the suite during development.

5. Test strategy

5.1 Levels

Level	What it covers	Tooling
Unit	<code>booking_service.py</code> business logic and the <code>admin_required</code> decorator, isolated from the real clock via <code>now=</code> parameters and, in one case, a genuine <code>unittest.mock</code> test double for <code>Booking.query</code>	pytest, in-memory SQLite, <code>unittest.mock</code>
Integration	Flask routes + real DB + Flask-Login auth together	pytest + Flask test client
System (E2E)	Full browser journeys	Selenium + Page Object Model
Acceptance	Manual walkthrough of the 12 E2E journeys by a group member before submission	Manual, checklist-based

5.2 Automation strategy

Selectors are `data-testid` attributes added to every interactive element during this pass specifically so E2E

tests target stable hooks instead of CSS classes or XPath that change with styling.

5.3 CI strategy

GitHub Actions runs unit -> integration -> E2E on every push/PR (E2E only after the first two jobs pass, since there's no point launching a browser if the business logic is already broken). Jenkins mirrors the same stages locally via Docker Compose for teams that want a local pipeline too.

6. Test environment

Python 3.12, SQLite (in-memory for unit/integration, a dedicated file for E2E since the app runs in a background thread). Headless Chrome for CI E2E runs.

7. Test data

tests/unit/conftest.py and tests/integration/conftest.py provide factory fixtures (make_user, make_equipment, make_booking) so every test builds exactly the data it needs rather than depending on shared, hand-created records. tests/e2e/conftest.py seeds a dedicated SQLite file for the live server fixture. seed.py remains for manual/demo use only and is never imported by the test suite.

8. Roles and responsibilities

(Fill in per group.)

Role	Responsibility
Application lead	app/models.py, app/routes.py, app/services/
Unit/integration test lead	tests/unit, tests/integration
E2E test lead	Page Objects, tests/e2e
CI/DevOps lead	GitHub Actions, Jenkins, Docker
Documentation lead	This document set

9. Entry criteria

- The application runs locally (python run.py) and seed.py succeeds.
- Any defect found during test-writing is fixed and the fix's regression test added before moving to the next test level (this is literally how DEF-002 was handled - discovered while writing IT-305, fixed immediately, suite re-run green before continuing).

10. Exit criteria

- 100% of written unit and integration tests pass (currently: 115/115).
- All 12 E2E journeys pass in CI with a real browser at least once before submission (not yet confirmed - see Test Summary Report).
- Branch coverage on app/services and app/auth.py $\geq 80\%$ (achieved: 99% and 100% respectively).
- No open defect of severity High or above (currently: 0 open, DEF-002 was High and is closed).
- All four PDF documents complete and consistent with the repository.

11. Risk-based prioritisation

Risk area	Likelihood	Impact	Priority	Mitigation
Approval logic double-counting a booking against itself (exactly DEF-002)	Was: certain (100% reproduction)	High	Highest	Fixed with exclude_booking_id; regression-tested by UT-013/IT-305; BVA-3 specifically covers the at-capacity boundary going forward
Double-booking / overbooking via date-overlap conflicts	Medium	High	High	UT-001 to UT-009, IT-304, decision tables DT-4/DT-8

Invalid state transitions reachable via a forged request	Low	High	High	Backend enforces <code>VALID_TRANSITIONS</code> regardless of UI state; E2E-011 forges a request directly against the route
Authorization bypass (student reaching admin actions)	Low	High	High	<code>admin_required</code> unit-tested directly (mocked <code>current_user</code>) plus IT-107/108/109/110
UI/template inconsistency (exactly DEF-001: two templates disagreeing on how to check admin status)	Was: certain	Medium	Medium	Fixed by adding <code>is_admin</code> as the single source of truth; IT-109/110 guard against regressing
CI-only flakiness (timing, browser setup)	Medium	Medium	Medium	Explicit Selenium waits only, dedicated ports/DB per E2E run

12. Schedule

Since this was a refinement of an existing codebase rather than a from-scratch build: defect audit and fixes first, then unit tests, then integration tests (which is what surfaced DEF-002), then E2E + Page Objects, then CI, then this documentation set.

13. Deliverables

Repository (application, tests/, .github/workflows/ci.yml, Jenkinsfile, docker/, README.md, docs/defect_log.md) plus four PDFs: this Test Plan, the Test Design Document, the Defect Log & Metrics document, and the Test Summary Report with the foundations reflection.

14. Traceability

See the Test Design Document's requirements table and per-technique mapping tables, and the README's project-structure section.